

LAPORAN PRAKTIKUM ABSTRACT CLASS & INTERFACE

Dibuat untuk memenuhi salah satu tugas Praktikum Mata Kuliah Pengembangan Berorientasi Objek



Dibuat Oleh:

NAMA : Rena Fiantina
NIM : 251511028
KELAS : 2-A
PROGRAM STUDI : D3 Teknik Informatika

**TEKNIK KOMPUTER DAN INFORMATIKA
POLITEKNIK NEGERI BANDUNG
2026**

A. Praktikum I – Employee, Manager, Sortable

a. Employee

Employee adalah class dengan state yang dibuat secara private yang berupa name, salary, hireDay, hireMonth dan hireYear. Class employee juga memiliki behavior seperti print(), raiseSalary(), hireYear() dan getSalary().

- Step 2 - Uji beberapa object Employee:

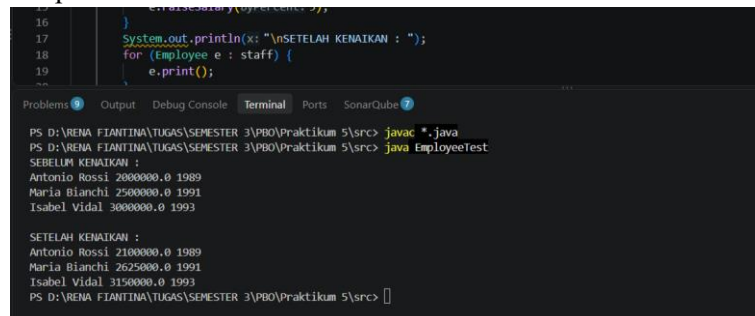
Dilakukan pengujian untuk melihat perubahan salary setelah method raiseSalary dipanggil. Dilakukan dengan membuat tiga object employee menggunakan class yang sama namun setiap object memiliki nilai state yang berbeda.

```
Employee[] staff = new Employee[3];

staff[0] = new Employee("Antonio Rossi", 2000000, 1, 10, 1989);
staff[1] = new Employee("Maria Bianchi", 2500000, 1, 12, 1991);
staff[2] = new Employee("Isabel Vidal", 3000000, 1, 11, 1993);

for (Employee e : staff) {
    e.raiseSalary(5);
}
```

Output :



```
16 }
17 System.out.println(x: "\nSETELAH KENAIKAN : ");
18 for (Employee e : staff) {
19     e.print();
...
Problems Output Debug Console Terminal Ports SonarQube
PS D:\RENA FIANTINA\TUGAS\SEMESTER 3\PBO\Praktikum 5> javac *.java
PS D:\RENA FIANTINA\TUGAS\SEMESTER 3\PBO\Praktikum 5> java EmployeeTest
SEBELUM KENAIKAN :
Antonio Rossi 2000000.0 1989
Maria Bianchi 2500000.0 1991
Isabel Vidal 3000000.0 1993

SETELAH KENAIKAN :
Antonio Rossi 2100000.0 1989
Maria Bianchi 2625000.0 1991
Isabel Vidal 3150000.0 1993
PS D:\RENA FIANTINA\TUGAS\SEMESTER 3\PBO\Praktikum 5> []
```

Dari output tersebut menunjukkan bahwa method yang sama dapat digunakan pada object yang berbeda dan sesuai dengan state masing-masing object. Sehingga setelah method raiseSalary(5) dijalankan, salary tiap object meningkat 5%.

b. Manager

Manager dibuat sebagai subclass dari Employee, sehingga class ini dapat mewarisi behavior dan state dari employee namun memiliki state tambahan berupa secretaryName. Kemudian method raiseSalary() di-override karena Manager punya aturan kenaikan salary yang berbeda.

Checkpoint:

1. Mengapa constructor Manager memanggil super(...)?

Karena Manager merupakan subclass dari Employee, sehingga super(...) digunakan untuk memanggil constructor Employee dan menginisialisasi state yang diwarisi dari Employee. Dengan begitu, Manager tidak perlu menulis ulang proses inisialisasi tersebut.

2. Mengapa Manager tetap memanfaatkan `super.raiseSalary(...)` daripada menulis ulang seluruh proses perubahan salary?

Karena proses dasar perubahan salary sudah dimiliki oleh Employee. Jadi Manager hanya perlu menghitung bonus tambahan berdasarkan lama kerja, kemudian meneruskan nilai persentasenya ke `raiseSalary()`. Dengan begitu, kode yang sudah ada di Employee tidak perlu ditulis ulang dan tetap digunakan.

- Step 4 - Uji Employee dan Manager dalam satu program

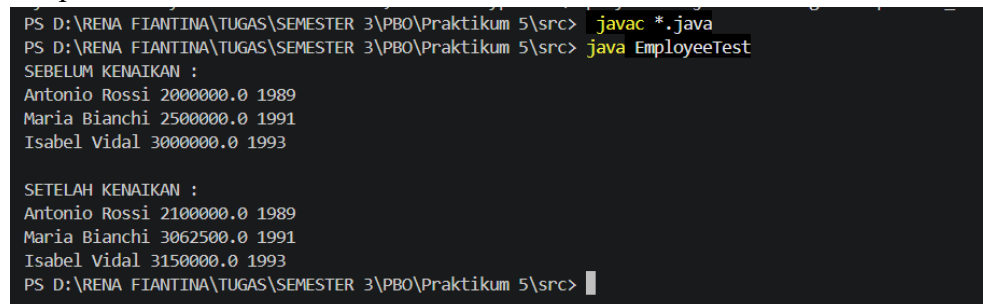
Dilakukan pengujian perhitungan method `raiseSalary` yang sudah di override dengan mengganti salah satu object pada array employee menjadi object manager.

```
public class EmployeeTest {
    public static void main(String[] args) {

        Employee[] staff = new Employee[3];

        staff[0] = new Employee("Antonio Rossi", 2000000,
1, 10, 1989);
        staff[1] = new Manager("Maria Bianchi", 2500000,
1, 12, 1991);
        staff[2] = new Employee("Isabel Vidal", 3000000,
1, 11, 1993);
```

Output



```
PS D:\RENA FIANTINA\TUGAS\SEMESTER 3\PBO\Praktikum 5\src> javac *.java
PS D:\RENA FIANTINA\TUGAS\SEMESTER 3\PBO\Praktikum 5\src> java EmployeeTest
SEBELUM KENAIKAN :
Antonio Rossi 2000000.0 1989
Maria Bianchi 2500000.0 1991
Isabel Vidal 3000000.0 1993

SETELAH KENAIKAN :
Antonio Rossi 2100000.0 1989
Maria Bianchi 3062500.0 1991
Isabel Vidal 3150000.0 1993
PS D:\RENA FIANTINA\TUGAS\SEMESTER 3\PBO\Praktikum 5\src> |
```

Ketika `raiseSalary(5)` dipanggil melalui variabel bertipe Employee, object Manager tetap meng-implementasikan `raiseSalary()` miliknya yang telah di-override.

c. Abstract Class Sortable

Dibuat sebagai abstract class yang berfungsi sebagai dasar untuk class yang dapat dibandingkan. Class ini mendeklarasikan abstract method `compare()`, tapi tidak menyediakan implementasi method tersebut. Karena merupakan abstract class, sortable tidak dapat dibuat menjadi object secara langsung.

Hasil Percobaan :

Dilakukan percobaan membuat object langsung dari abstract class menggunakan `new Sortable()`. Compiler menolak proses tersebut karena Sortable merupakan abstract class.

```
PS D:\RENA FIANтина\TUGAS\SEMESTER 3\PBO\Praktikum 5\src> javac Sortable.java EmployeeTest.java
EmployeeTest.java:22: error: Sortable is abstract; cannot be instantiated
    Sortable s = new Sortable();
                    ^
1 error
PS D:\RENA FIANтина\TUGAS\SEMESTER 3\PBO\Praktikum 5\src> |
```

Checkpoint 4 : *Sortable berguna sebagai class untuk menetapkan kewajiban compare(...), tetapi tidak memiliki bentuk object konkret yang perlu dibuat langsung. Apa bukti compiler yang Anda dapatkan?*

⇒ Bukti dari compiler adalah munculnya error ketika mencoba membuat object menggunakan `Sortable s = new Sortable();`. Compiler menolak karena `Sortable` merupakan abstract class sehingga tidak dapat diinstansiasi secara langsung.

d. Employee Implements Compare

Setelah employee dibuat menjadi subclass dari sortable, employee wajib memnuhi abstract method `compare()` yang dideklarasikan oleh sortable.

```
public class Employee extends Sortable {
    ...

    @Override
    public int compare(Sortable other) {
        Employee otherEmployee = (Employee) other;

        if (getSalary() < otherEmployee.getSalary()) return -1;
        if (getSalary() > otherEmployee.getSalary()) return 1;
        return 0;
    }
}
```

Hasil Percobaan :

Ketika Employee sudah menjadi subclass tetapi belum memiliki `compare()`, compiler menghasilkan error karena Employee belum memenuhi abstract method dari Sortable.

```
PS D:\RENA FIANтина\TUGAS\SEMESTER 3\PBO\Praktikum 5\src> javac Sortable.java Employee.java Manager.java EmployeeTest.java
Employee.java:1: error: Employee is not abstract and does not override abstract method compare(Sortable) in Sortable
public class Employee extends Sortable {
    ^
1 error
PS D:\RENA FIANтина\TUGAS\SEMESTER 3\PBO\Praktikum 5\src> |
```

Checkpoint 5 : *Apa yang diwajibkan Sortable kepada Employee? Mana bagian yang hanya deklarasi pada superclass abstract, dan mana bagian yang menjadi implementasi konkret di Employee?*

⇒ Sortable mewajibkan Employee memiliki method `compare(Sortable other)`. Pada Sortable, `compare()` hanya berupa deklarasi abstract tanpa implementasi. Pada Employee, method tersebut diimplementasikan secara konkret dengan membandingkan nilai salary dan menghasilkan -1, 0, atau 1.

e. Hasil Compare

Method `compare()` digunakan untuk membandingkan salary antara object `Employee`. Method mengembalikan -1 jika salary object pertama lebih kecil, 1 jika lebih besar, dan 0 jika keduanya sama.

```
System.out.println("staff[0] vs staff[1] : "
    + staff[0].compare(staff[1]));

System.out.println("staff[1] vs staff[2] : "
    + staff[1].compare(staff[2]));

System.out.println("staff[2] vs staff[0] : "
    + staff[2].compare(staff[0]));
```

Output :

```
PS D:\RENA_FIANTINA\TUGAS\SEMESTER 3\PBO\Praktikum 5\src> javac Sortable.java Employee.java Manager.java EmployeeTest.java
PS D:\RENA_FIANTINA\TUGAS\SEMESTER 3\PBO\Praktikum 5\src> java EmployeeTest
staff[0] vs staff[1] : -1
staff[1] vs staff[2] : -1
staff[2] vs staff[0] : 1
```

f. Inheritance Chain

Inheritance pada percobaan ini membentuk rantai `Sortable` → `Employee` → `Manager`. `Employee` mewarisi `Sortable` dan mengimplementasikan method `compare()`. `Manager` kemudian mewarisi `Employee` sehingga `Manager` juga dapat menggunakan `compare()` tanpa harus menuliskan `extends Sortable` lagi.

Eksperimen B - tanpa menuliskan 'Extends Sortable' pada `Manager`

```
Manager m = new Manager("Maria Bianchi", 2500000, 1, 12, 1991);
Employee e = new Employee("Isabel Vidal", 3000000, 1, 11, 1993);

System.out.println("\nINHERITANCE CHAIN ");
System.out.println("Manager compare Employee: " + m.compare(e));
```

Output :

```
PS D:\RENA_FIANTINA\TUGAS\SEMESTER 3\PBO\Praktikum 5\src> javac *.java
PS D:\RENA_FIANTINA\TUGAS\SEMESTER 3\PBO\Praktikum 5\src> java EmployeeTest
staff[0] vs staff[1] : -1
staff[1] vs staff[2] : -1
staff[2] vs staff[0] : 1

SEBELUM KENAIKAN :
Antonio Rossi 2000000.0 1989
Maria Bianchi 2500000.0 1991
Isabel Vidal 3000000.0 1993

SETELAH KENAIKAN :
Antonio Rossi 2100000.0 1989
Maria Bianchi 3062500.0 1991
Isabel Vidal 3150000.0 1993

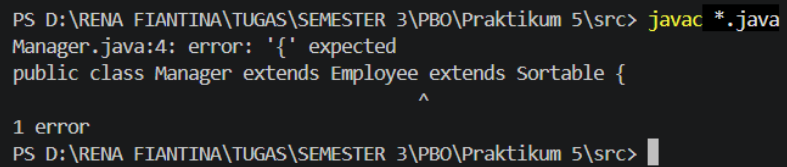
INHERITANCE CHAIN
Manager compare Employee: -1
PS D:\RENA_FIANTINA\TUGAS\SEMESTER 3\PBO\Praktikum 5\src> |
```

Walaupun Manager tidak menuliskan extends Sortable, object Manager tetap bisa menggunakan compare(). Hal itu dikarenakan Manager mewarisi Employee, sedangkan Employee mewarisi Sortable.

Eksperimen A – Multiple Extends

```
public class Manager extends Employee extends Sortable {  
}
```

Output :



```
PS D:\RENA FIANTINA\TUGAS\SEMESTER 3\PBO\Praktikum 5\src> javac *.java  
Manager.java:4: error: '{' expected  
public class Manager extends Employee extends Sortable {  
                                     ^  
1 error  
PS D:\RENA FIANTINA\TUGAS\SEMESTER 3\PBO\Praktikum 5\src> |
```

Hanya diperbolehkan satu superclass langsung pada sebuah class. Oleh karena itu, Manager cukup menggunakan extends Employee, karena Employee sendiri sudah extends Sortable.

Checkpoint 6 : Mengapa Manager tetap memiliki compare(...) walaupun tidak pernah menulis `extends Sortable`? Gambarkan rantai class: Sortable <- Employee <- Manager

⇒ Method compare() didefinisikan pada Sortable dan diimplementasikan oleh Employee. Karena Manager merupakan subclass dari Employee, maka Manager dapat menggunakan method compare() yang diwariskan dari Employee.

B. Praktikum II – Goods & Taxable

a. Goods

Goods merupakan class yang menyimpan informasi barang berupa description dan price. Kedua attribute dibuat private, sedangkan getDescription() dan getPrice() digunakan untuk mengambil nilainya.

Prediksi singkat :

- State apa yang cukup umum sehingga berada di Goods?
⇒ description dan price, karena keduanya merupakan informasi dasar yang dimiliki semua barang.
- State apa yang hanya dimiliki Food, Toy, dan Book?
⇒ Masing-masing subclass memiliki state/atribut khusus sesuai jenis barangnya, yang tidak dimiliki Goods secara umum.
- Mengapa Taxable tidak diterapkan pada semua subclass Goods pada model ini?
⇒ Karena tidak semua jenis barang dianggap memiliki perilaku/perhitungan pajak yang sama. Taxable hanya diterapkan pada class yang memang membutuhkan kontrak perhitungan pajak.

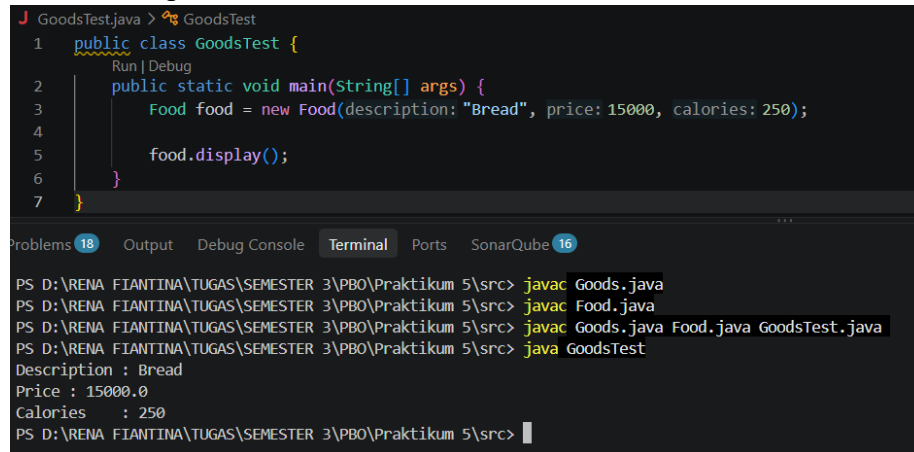
Checkpoint 7 : Mengapa description dan price diletakkan di Goods, bukan ditulis ulang di Food, Toy, dan Book?

⇒ Karena description dan price merupakan state yang umum dimiliki semua jenis Goods. Dengan disimpan di Goods, Food, Toy, dan Book cukup mewarisinya sehingga tidak perlu menulis ulang atribut yang sama di setiap subclass.

b. Food

Food merupakan subclass dari Goods yang memiliki state tambahan berupa calories. Food mewarisi description, price, serta display() dari Goods.

Code & Output :



```
1 public class GoodsTest {
2     public static void main(String[] args) {
3         Food food = new Food(description: "Bread", price: 15000, calories: 250);
4
5         food.display();
6     }
7 }
```

```
PS D:\RENA FIANTINA\TUGAS\SEMESTER 3\PBO\Praktikum 5\src> javac Goods.java
PS D:\RENA FIANTINA\TUGAS\SEMESTER 3\PBO\Praktikum 5\src> javac Food.java
PS D:\RENA FIANTINA\TUGAS\SEMESTER 3\PBO\Praktikum 5\src> javac Goods.java Food.java GoodsTest.java
PS D:\RENA FIANTINA\TUGAS\SEMESTER 3\PBO\Praktikum 5\src> java GoodsTest
Description : Bread
Price : 15000.0
Calories : 250
PS D:\RENA FIANTINA\TUGAS\SEMESTER 3\PBO\Praktikum 5\src>
```

Food menggunakan super(description, price) untuk menginisialisasi state yang diwarisi dari Goods. Method display() di-override dan menggunakan super.display() agar informasi description dan price tetap ditampilkan, kemudian menambahkan calories.

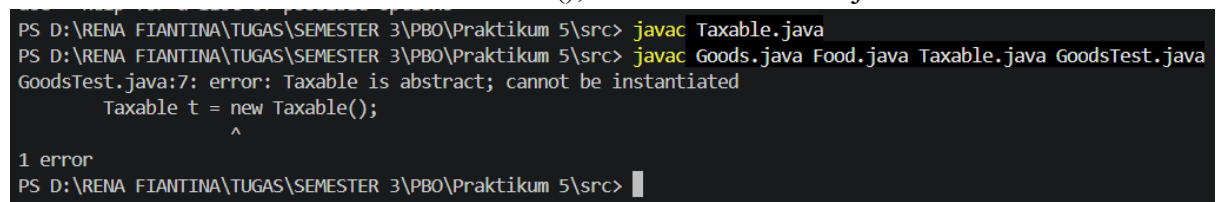
c. Interface Taxable

Taxable merupakan interface yang mendefinisikan kontrak berupa calculateTax(). Class yang mengimplementasikan Taxable harus menyediakan method tersebut.

```
public interface Taxable {
    double taxRate = 0.06;
    double calculateTax();
}
```

Percobaan & Output yang muncul:

Menguji apakah interface taxable bisa dibuat menjadi object secara langsung dengan memasukan kode Taxable t = new Taxable(); kedalam GoodsTest.java



```
PS D:\RENA FIANTINA\TUGAS\SEMESTER 3\PBO\Praktikum 5\src> javac Taxable.java
PS D:\RENA FIANTINA\TUGAS\SEMESTER 3\PBO\Praktikum 5\src> javac Goods.java Food.java Taxable.java GoodsTest.java
GoodsTest.java:7: error: Taxable is abstract; cannot be instantiated
    Taxable t = new Taxable();
                  ^
1 error
PS D:\RENA FIANTINA\TUGAS\SEMESTER 3\PBO\Praktikum 5\src>
```

Compiler menolak new Taxable() karena Taxable merupakan interface. Interface ini berfungsi sebagai contract yang nantinya harus dipenuhi oleh class yang menggunakan implements Taxable.

Checkpoint 8 : Apa persamaan hasil eksperimen `new Sortable()` dan `new Taxable()`? Apa perbedaan bentuk deklarasi Sortable dan Taxable?

⇒ Keduanya sama-sama tidak dapat diinstansiasi secara langsung, tetapi Sortable berbentuk abstract class, sedangkan Taxable berbentuk interface. Sortable nantinya diwarisi menggunakan extends, sedangkan Taxable dipenuhi oleh class menggunakan implements.

d. Toy

Toy merupakan subclass dari Goods yang sekaligus mengimplementasikan interface Taxable. Toy memiliki state khusus minimumAge dan wajib menyediakan calculateTax() karena adanya contract Taxable.

```
public class Toy extends Goods implements Taxable {
    private int minimumAge;

    public Toy(String description, double price, int minimumAge)
    {
        super(description, price);
        this.minimumAge = minimumAge;
    }

    public int getMinimumAge() {
        return minimumAge;
    }

    @Override
    public double calculateTax() {
        return getPrice() * taxRate;
    }

    @Override
    public void display() {
        super.display();
        System.out.println("Minimum age : " + minimumAge);
        System.out.println("Tax : " + calculateTax());
    }
}
```

extends Goods membuat Toy mewarisi state dan method dari Goods, sedangkan implements Taxable membuat Toy wajib menyediakan calculateTax().

Checkpoint 9: Pada deklarasi `Toy extends Goods implements Taxable`, bagian mana yang menyatakan "Toy adalah Goods" dan bagian mana yang menyatakan "Toy memenuhi kontrak Taxable"?

⇒ extends Goods menunjukkan hubungan inheritance bahwa Toy adalah Goods, sedangkan implements Taxable menunjukkan bahwa Toy memenuhi contract Taxable dengan menyediakan calculateTax().

e. Book

Book merupakan subclass dari Goods yang mengimplementasikan interface Taxable. Book memiliki state khusus berupa author dan menyediakan implementasi calculateTax() sesuai contract Taxable.

Perbandingan Toy dan Book :

⇒ Toy dan Book memiliki parent class yang sama, yaitu Goods, dan sama-sama mengimplementasikan interface Taxable. Namun, keduanya memiliki state spesifik yang berbeda. Toy memiliki state minimumAge, sedangkan Book memiliki state author. Keduanya tetap memiliki description dan price yang diwarisi dari Goods.

Aspek	Toy	Book
Parent	Goods	Goods
Contract	Taxable	Taxable
State khusus	minimumAge	author
State dari Goods	description, price	description, price

f. Extends + Implements

Pengujian dilakukan dengan membuat object Food, Toy, dan Book. Ketiganya merupakan subclass dari Goods, tetapi hanya Toy dan Book yang mengimplementasikan Taxable

Code :

```
Taxable taxableToy = toy;
Taxable taxableBook = book;

System.out.println("Toy tax : " + taxableToy.calculateTax());
System.out.println("Book tax : " + taxableBook.calculateTax());
```

Output :

```
PS D:\RENA FIANTINA\TUGAS\SEMESTER 3\PBO\Praktikum 5\src> java GoodsTest
Description : Bread
Price : 15000.0
Calories    : 250

Description : Robot
Price : 120000.0
Minimum age : 8
Tax : 7200.0

Description : Java Basics
Price : 90000.0
Author : A. Programmer
Tax : 5400.0

Toy tax : 7200.0
Book tax : 5400.0
PS D:\RENA FIANTINA\TUGAS\SEMESTER 3\PBO\Praktikum 5\src> 
```

Food, Toy, dan Book dapat menggunakan state dan behavior dari Goods melalui inheritance. Toy dan Book juga dapat direferensikan sebagai Taxable karena keduanya mengimplementasikan interface tersebut. Perhitungan tax menggunakan $\text{price} \times \text{taxRate}$, sehingga Toy menghasilkan 7200.0 dan Book menghasilkan 5400.0. Food tidak memiliki contract Taxable pada model ini.

g. Sengaja langgar kontrak Interface

Method calculateTax() dihapus sementara dari Toy, tetapi deklarasi implements Taxable tidak diubah.

Compiler meminta Toy untuk mengimplementasikan method calculateTax() karena Toy menggunakan implements Taxable. Method tersebut merupakan contract yang wajib dipenuhi oleh class konkret yang mengimplementasikan Taxable.

Hasil Percobaan :

```
PS D:\RENA FIANTINA\TUGAS\SEMESTER 3\PBO\Praktikum 5\src> javac Goods.java Food.java Taxable.java Toy.java Book.java GoodsTest.java
Toy.java:1: error: Toy is not abstract and does not override abstract method calculateTax() in Taxable
public class Toy extends Goods implements Taxable {
      ^
Toy.java:17: error: cannot find symbol
    System.out.println("Tax : " + calculateTax());
                                ^
    symbol:   method calculateTax()
    location: class Toy
2 errors
PS D:\RENA FIANTINA\TUGAS\SEMESTER 3\PBO\Praktikum 5\src>
```

Checkpoint 10 : Keyword implements bukan sekadar label. Ketika sebuah class implements interface, compiler menuntut class konkret tersebut memenuhi method yang dideklarasikan interface.

C. Perbandingan Abstract Class dan Interface Berdasarkan Pengalaman Code

Pertanyaan	Sortable (Abstract Class)	Taxable (Interface)
Dapat dibuat object langsung?	Tidak. Saat mencoba new Sortable() mendapatkan compile error.	Tidak. ketika mencoba new Taxable() mendapatkan compile error.
Menggunakan extends atau implements?	extends. Employee extends Sortable.	implements. Toy dan Book implements Taxable.
Peran utamanya pada kasus praktikum?	Menjadi parent yang mewajibkan Employee menyediakan compare().	Menjadi contract yang mewajibkan Toy dan Book menyediakan calculateTax().

Apakah class dapat mempunyai parent class lain sekaligus?	Tidak, berdasarkan percobaan Manager extends Employee extends Sortable menghasilkan error.	Bisa sekaligus dengan satu parent class, dibuktikan Toy extends Goods implements Taxable dan Book extends Goods implements Taxable.
---	--	---

D. Table Skenario Pengujian

ID	Skenario	Expected Result	Hasil
T5-01	new Sortable()	Compiler menolak	Error
T5-02	Employee tanpa compare()	Compiler menuntut implementasi	Error
T5-03	compare()	-1/0/1 sesuai salary	Sesuai
T5-04	Manager hanya extends Employee	Tetap punya compare()	Sesuai
T5-05	Multiple extends	Compiler menolak	Error
T5-06	Toy/Book implements Taxable	calculateTax() dapat dipanggil	Sesuai
T5-07	calculateTax() dihapus	Compiler menolak	Error
T5-08	Food sebagai Goods	State/behavior Goods tersedia	Sesuai

E. Refleksi

a. Pengalaman praktikum

1. Mengapa Sortable berguna walaupun object Sortable tidak pernah dibuat?
 - ⇒ Sortable berguna sebagai abstract class yang menjadi dasar bagi class turunannya. Walaupun object Sortable tidak dibuat, Sortable menetapkan bahwa subclass seperti Employee harus memiliki method compare(). Hal ini terbukti ketika Employee belum memiliki compare(), compiler menghasilkan error.
2. Apa yang terjadi ketika Employee extends Sortable tetapi belum membuat compare()?
 - ⇒ Compiler menghasilkan error karena Employee merupakan class konkret yang mewarisi abstract method compare() dari Sortable, tetapi belum menyediakan implementasinya. Setelah compare() ditambahkan ke Employee, program dapat dikompilasi kembali.
3. Mengapa Manager tetap dapat menggunakan compare() tanpa menulis extends Sortable?
 - ⇒ Karena Manager mewarisi Employee, sedangkan Employee mewarisi Sortable. Jadi compare() dapat diteruskan melalui inheritance chain:
Sortable → Employee → Manager
4. Mengapa class Manager extends Employee extends Sortable tidak diperlukan dan tidak valid di Java?

⇒ Tidak diperlukan karena Employee sudah mewarisi Sortable, sehingga Manager otomatis berada dalam inheritance chain tersebut. Percobaan Manager extends Employee extends Sortable juga menghasilkan error karena sebuah class tidak dapat memiliki lebih dari satu superclass langsung melalui extends

5. Apa yang dijanjikan Toy ketika menulis implements Taxable?

⇒ Toy menyatakan bahwa class tersebut memenuhi contract Taxable. Karena itu, Toy wajib menyediakan method calculateTax(). Ketika method tersebut dihapus tetapi implements Taxable tetap ada, compiler menghasilkan error.

6. Mengapa Goods lebih tepat menjadi superclass bagi Food/Toy/Book, sedangkan Taxable lebih tepat menjadi interface pada model ini?

⇒ Goods lebih cocok menjadi superclass karena Food, Toy, dan Book sama-sama memiliki description dan price. Sedangkan Taxable hanya dibutuhkan oleh Toy dan Book untuk menghitung pajak, sehingga lebih cocok digunakan sebagai interface.

7. Apa bukti dari compiler bahwa interface benar-benar merupakan contract?

⇒ Buktinya terlihat ketika calculateTax() dihapus dari Toy tetapi Toy tetap implements Taxable. Compiler menolak Toy karena belum memenuhi method yang diwajibkan oleh Taxable. Jadi implements Taxable bukan sekadar penanda, tetapi memiliki kewajiban yang diperiksa oleh compiler.

8. Abstract class membantu saya ketika ada class yang ingin dijadikan dasar untuk beberapa subclass dan ada method yang wajib dibuat oleh subclass tersebut. Sedangkan interface membantu saya ketika beberapa class perlu memiliki contract atau method tertentu, seperti calculateTax() pada Toy dan Book.